THE COMPLETE PROMPT ENGINEERING RULEBOOK

A Comprehensive Guide to Prompting GPT-5.6 Luna

Version 1.0 | Target model: GPT-5.6 Luna | Prompt engineering handbook

Purpose

Maximise response quality, accuracy, efficiency and task completion through clear objectives, relevant context, precise constraints, appropriate tools and meaningful verification.

Central principle

Do not write the longest possible prompt. Communicate the complete requirement with the least unnecessary complexity. Define the outcome, provide the information that materially affects it, establish boundaries and explain how success will be evaluated.

This 14-page edition condenses the original 47-page rulebook. It retains its core rules, eight-part framework, research and domain-specific guidance, reusable templates, quality checks and API-setting explanations while removing repeated examples and overlapping explanations.

1. Understanding Prompt Engineering

What a prompt must communicate

Prompt engineering is the process of designing, structuring and refining instructions to obtain a desired result. A prompt is more than a question; it communicates:

• What the model must accomplish and why the task matters.

• Which information, files, sources and resources it should use.

• Which limitations, priorities and permissions it must respect.

• What the final result should contain and look like.

• What conditions determine whether the task is complete.

GPT-5.6 Luna: practical prompting principles

1. Define the desired outcome instead of scripting every operation.

2. Supply relevant context and supporting materials.

3. State important constraints clearly and distinguish them from preferences.

4. Define success criteria and completion conditions.

5. Avoid repeating the same instruction.

6. Specify when the model should ask, proceed, seek approval or stop.

7. Keep prompts concise without removing important information.

Three fundamentals

Principle	Meaning
Clarity	The model understands exactly what is requested.
Completeness	Sufficient information is available without unnecessary assumptions.
Precision	Important details, priorities and expected output are unmistakable.

Luna is described in the supplied model information as a cost-sensitive, high-volume model, approximately corresponding to a nano tier in earlier GPT-5 model families. The practical lesson is to provide clear, outcome-oriented instructions and avoid unnecessary complexity.

2. Fundamental Rules of Prompting

Define the objective

Begin with the actual task and desired result. Avoid vague requests such as “help with programming” or “make my project better.” Use action-oriented verbs that describe the work: explain, analyse, compare, evaluate, design, develop, debug, investigate, summarise, research, recommend, implement, optimise, verify, extract or classify.

Provide relevant context

Context prevents a technically correct but unsuitable answer. For a software task, include experience level, technology stack, application purpose, intended users, development environment, resources, restrictions and existing implementation. Include only details that can influence the answer.

Be specific and descriptive

Where relevant, define subject, scope, depth, audience, desired result, format and restrictions. For teaching, state the exact syllabus topics, learner level, concepts, examples and distinctions to cover.

Right-size the request

• Divide complex work into logical stages when this improves manageability, accuracy or error detection.

• For development, stages may include requirements, architecture, database, backend, frontend, integration, testing and deployment.

• Do not force a multi-stage workflow for a straightforward task; let the model choose an efficient method when the outcome is clear.

Prioritise and measure

Separate mandatory requirements from preferences. Replace “short”, “detailed”, “professional” or “optimised” with measurable criteria such as a word limit, exact number of examples, defined coverage, formal tone or complexity analysis.

3. Structuring Effective Prompts

Put instructions before supporting context

Place the main task and requirements near the beginning. Clearly separate instructions from the source material to be processed. For long source-heavy tasks, use labelled sections so the model can distinguish the task from the supplied content.

Use delimiters and headings

• Useful delimiters include ###, triple quotation marks, horizontal separators and XML-style tags.

• Useful headings: ROLE, OBJECTIVE, CONTEXT, AVAILABLE MATERIALS, REQUIREMENTS, CONSTRAINTS, SOURCES, SUCCESS CRITERIA, OUTPUT FORMAT, VERIFICATION and STOP CONDITIONS.

• Use only headings that materially help the task; not every prompt needs every section.

Example structure
TASK: Analyse the supplied dataset and identify trends.
REQUIREMENTS: Inspect columns, calculate important measures, identify limitations.
DATA: [Insert dataset inside clear delimiters.]
OUTPUT: Dataset overview, findings, numerical evidence, limitations and conclusion.

Define the output format

Specify whether you want paragraphs, bullets, numbered steps, tables, technical documentation, source code, JSON, CSV, mathematical derivations, step-by-step explanations or an executive summary. For tables, name the required columns and request a concise recommendation when appropriate.

Use examples only when useful

Few-shot examples are useful for unusual output structures, custom classification rules, consistent extraction, repeated transformations and code generation patterns. Provide representative examples only when the required pattern is difficult to describe. Do not add examples when the expected output is already obvious.

Treat embedded content as data

When analysing a document, dataset or quoted text, explicitly state that instructions found inside that material are content to analyse, not commands to follow.

4. Advanced Prompting Techniques

Technique	When and how to use it
Zero-shot	Start with a direct instruction and no examples for ordinary tasks.
Few-shot	Add a few input-output examples when formatting, classification or consistency rules are unusual.
Task decomposition	Break complex objectives into manageable components; integrate and test them afterward.
Comparative prompting	Specify decision criteria such as cost, complexity, performance, security, scalability and suitability.
Verification prompting	Ask for checks of logic, requirements, edge cases, calculations and evidence.
Iterative prompting	Refine a prior result with targeted follow-ups instead of repeating the entire request.
Explicit assumptions	Ask the model to identify missing information, assess its impact, ask if material, or proceed with a stated reasonable assumption if minor.

Use negative instructions carefully

Pair prohibitions with a positive alternative. Instead of “do not use complicated language,” specify “use simple technical language for a second-year engineering student and explain necessary terminology.” Instead of “avoid unnecessary libraries,” specify “prefer the standard library and add dependencies only when they provide a clear advantage.”

Leading words for code

Prefixes such as `import`, `SELECT` or `` may help establish an output type, but they are optional. A clear language and output instruction is usually sufficient.

Assumption-handling instruction

If information is missing, identify it and assess whether it materially affects the result. Ask a targeted question when it does. Otherwise proceed with a reasonable assumption and clearly state any assumption that affects the conclusion.

5. The Eight-Part Prompt Framework

Use this framework for demanding tasks. Not every prompt needs every component.

Component	What to specify
1. ROLE	Relevant function or expertise: software engineer, instructor, analyst, cybersecurity specialist, technical writer. Avoid fictional qualifications or irrelevant titles.
2. PERSONALITY & COLLABORATION	Tone and working style: direct, professional, accessible, analytical, evidence-oriented and constructively critical. Ask only essential questions; do not simply agree.
3. GOAL	The exact final result. This is the most important component.
4. SUCCESS CRITERIA	Measurable conditions that must be satisfied before completion.
5. CONSTRAINTS	Technology, budget, time, resources, preservation requirements and prohibited changes.
6. TOOLS & SOURCES	Relevant official documentation, primary research, supplied files, datasets, existing project files and verification resources.
7. OUTPUT	Deliverable type, structure, audience, tone and length.
8. STOP RULES	When to ask, proceed, seek approval, disclose a limitation or stop after success criteria are met.

Useful stop rules

• Ask before irreversible changes or consequential external actions.

• Do not publish, submit, purchase or send without confirmation.

• Ask when essential information is missing.

• Disclose unverifiable sources or requirements that cannot be implemented.

• If an operation is unavailable, explain what failed and offer the closest feasible alternative.

• Stop when all success criteria are satisfied; do not continue adding unnecessary work.

6. Research and Information Accuracy

Define research scope

State the exact question, geography and time period. Geographic scope may be global, India, Karnataka, coastal Karnataka or a specific city. For changing information, specify a date range or publication window and require a currentness check.

Prioritise sources

1. Government publications and official statistics.

2. Official institutional documentation.

3. Peer-reviewed research papers.

4. University publications.

5. Established industry reports.

6. Reputable news publications.

7. Expert commentary.

8. Forums and community discussions.

Choose sources appropriate to the subject. Official data is important for regulatory and statistical questions; community discussions can help understand practical experiences.

Require evidence for important claims

• State each major finding and provide supporting evidence and source.

• Include publication dates where relevant and distinguish verified information from estimates.

• Do not invent sources, citations, statistics or quotations.

• Identify conflicting findings, uncertainty, limitations, alternative explanations and possible bias.

• Include evidence against a conclusion, not only evidence supporting it.

Separate information types

Type	Meaning
Verified fact	Directly supported by reliable evidence.
Estimate	Approximation or calculation based on stated assumptions.
Opinion	Judgement or interpretation.
Recommendation	Proposed action based on available evidence.

Research quality depends on evidence and coverage, not response length. Define the specific factors to analyse and request numerical evidence where available.

7. Programming and Software Development

Specify the environment

• Programming language and version, operating system, IDE and development environment.

• Frameworks, libraries, database and existing project structure.

• Required code level: beginner-friendly, minimal, readable, production-oriented, optimised, object-oriented, functional or complete application.

Protect existing projects

Provide relevant files or repository structure. State exactly which files may change, which must remain unchanged, existing database structures and records, functionality to preserve and new functionality required. Request the smallest reliable change and prohibit unrelated modifications.

Request complete usable code

When a complete implementation is requested, specify required imports, complete code, file placement, installation commands, execution commands, expected output and important implementation notes. Do not accept fragments when a complete solution is required.

Error handling and debugging

Define handling for empty or invalid inputs, missing files, incorrect formats, duplicates, database errors, network failures and unexpected user actions. For debugging, provide existing code, exact error or traceback, expected behaviour, actual behaviour, environment and relevant recent changes.

Verification and complexity

• Check syntax, variable names, imports, database interactions, edge cases, validation and compatibility.

• Where execution is available, run actual tests; code inspection alone does not prove functionality.

• Never claim code was executed or tested unless it was.

• For algorithmic tasks, request time and space complexity, including best, average and worst cases when relevant. Do not demand it for trivial exercises.

8. Academic Learning, Data and Mathematics

Academic learning

• Specify knowledge level, university, course, semester, subject, syllabus, module and examination requirements.

• Define teaching method: introduction, fundamentals, detailed explanation, working principles, examples, applications, distinctions and summary.

• Build prerequisites progressively; identify missing foundational knowledge before advancing.

• Specify exam needs: conceptual understanding, MCQ notes, descriptive answers, numericals, university or competitive exams.

• Set depth: beginner, intermediate, advanced or research level. Avoid “explain everything” without scope.

Data analysis

• Describe dataset name, file format, record count if known, columns, source, objective, assumptions and deliverables.

• Specify inspection, missing values, duplicates, data types, cleaning, statistics, patterns, visualisations and conclusions.

• Choose data treatment appropriately; do not assume mean imputation suits categorical or highly skewed data.

• Request actual numerical values, calculation periods and limitations. Never invent results.

Mathematical reasoning

1. Identify given information and the required result.

2. State the applicable formula and assumptions.

3. Show substitution and calculation clearly.

4. Give the final answer with units where applicable.

5. Check the result, domain restrictions and precision.

6. Request alternative methods when they add value; compare their complexity and suitability.

For advanced problems, explicitly request independent verification and identify conditions under which the solution is valid.

9. Business Decisions, Efficiency and Action Control

Business and finance

Define the decision, market, geography, target customer, budget, available working hours, resources and risk tolerance. Request quantitative analysis where relevant:

• Initial investment, fixed and variable costs, revenue, gross and net profit.

• Break-even point, return on investment and payback period.

• Optimistic, base and pessimistic scenarios with explicit assumptions.

• Feasibility, downside risk and justified recommendation—not just highest potential revenue.

Efficiency and resource management

• Keep the prompt as short as possible while complete; never remove essential requirements just to reduce length.

• Avoid repeated or unnecessary examples; start zero-shot and add few-shot examples only for genuine ambiguity.

• Describe outcomes instead of prescribing every internal step.

• Use follow-up prompts that refer to the relevant earlier decision or component.

• Escalate detail only when the initial result is insufficient; prompt length cannot compensate for model limitations.

Tools, sources and permissions

• Request external research only for current or externally verifiable information; supplied-text tasks, simple calculations and creative writing may not need web research.

• Specify which document is authoritative and how conflicts should be reported.

• Define allowed actions and require confirmation before deleting, publishing, submitting, purchasing or sending.

• If an operation is unavailable, explain the limitation, identify what was not completed and offer a feasible alternative. Never claim success falsely.

10. Common Prompting Mistakes and Universal Template

Mistake	Better approach
Vague objective / missing context	Define outcome and supply relevant background.
Too many unrelated tasks	Divide into logical components.
Repeated instructions	Consolidate overlapping requirements.
Undefined output	Specify structure, format and length.
Imprecise constraints	Use measurable restrictions.
Unsupported assumptions	Require explicit assumptions and uncertainty.
Excessive examples	Use representative examples only.
No success criteria	Define measurable completion conditions.
No source requirements	Specify source quality and evidence.
No error handling	Define failure cases and responses.
Overly rigid workflow	Specify outcome; prescribe steps only when needed.
Blind acceptance	Verify important outputs.
Repeating conversation context	Refer to relevant prior decisions and restate critical constraints when needed.

Universal master prompt

ROLE: Act as [relevant expert].
COLLABORATION: Use [tone/style]. Be objective, precise and constructive. Ask only when essential information is missing.
GOAL: [Exact outcome.]
CONTEXT: [Relevant background and previous decisions.]
MATERIALS: [Files, datasets, code and source priority.]
REQUIREMENTS: [Numbered mandatory requirements.]
SUCCESS CRITERIA: [Measurable completion conditions.]
CONSTRAINTS: [Technical, budget, time, preservation and prohibited changes.]
SOURCES/TOOLS: [Required sources, tools and verification.]
OUTPUT: [Format, structure, length, audience and tone.]
STOP RULES: Ask when essential information is missing; state minor assumptions; obtain approval before consequential actions; disclose limitations.

11. Ready-to-Use Prompt Templates

Academic learning

Act as an experienced instructor in [subject]. My level is [level]. Teach [topic] according to [course/syllabus]. Explain progressively; cover definitions, distinctions, examples, terminology, derivations where relevant and practical applications. Output organised study material. Avoid unrelated topics and commentary.

Programming / project development

Act as a software engineer specialising in [language/framework]. Objective: [task]. Environment: [OS, IDE, versions, framework, database]. Requirements: readable code, specified stack, input validation, edge-case handling and preservation of existing functionality. Constraints: [files allowed, files unchanged, resources]. Deliver complete implementation, file placement, installation and execution instructions, and testing guidance. Verify as far as available tools permit.

Debugging

Problem: [issue]. Expected behaviour: [expected]. Actual behaviour: [actual]. Environment: [details]. Code: [code]. Error: [traceback]. Identify root cause and smallest reliable correction. Preserve unrelated code. Explain cause, correction and how to verify it.

Research

Research objective: [question]. Scope: [geography and period]. Prioritise official and primary sources, then peer-reviewed research and reputable secondary publications. Present verified facts, numerical evidence, trends, conflicting findings, uncertainty and limitations. Provide supporting citations and a clear conclusion.

Data analysis

Analyse [dataset] using [tools]. Objective: [question]. Inspect structure, missing and duplicate values, data types and quality. Clean appropriately, perform relevant statistics, identify patterns, create useful visualisations and report numerical findings. Explain methodology, assumptions and limitations. Do not invent results.

Comparison / decision

Compare [options] in the context of [decision]. Evaluate [criteria]. Use current reliable information where necessary. Present advantages, disadvantages, quantitative differences, risks and a recommendation justified against the stated criteria.

12. Quality Control and API Parameters

Prompt quality checklist

Area	Questions to check
Objective	Is the task and exact result clear? Is the scope specific?
Context	Is relevant background, knowledge level and source material supplied?
Instructions	Are requirements clear, prioritised and non-contradictory?
Constraints	Are technical, budget, time, preservation and permission limits stated?
Evidence	Are source quality, verification, facts versus estimates and citations defined?
Output	Are format, detail, audience and mandatory sections specified?
Completion	Are success criteria, clarification policy, stop rules and final checks defined?

Model settings and API parameters

This section applies primarily to developers using the OpenAI API. Do not confuse API configuration with ordinary ChatGPT conversation instructions.

Setting	Meaning and practical guidance
Model selection	Affects capability, latency, cost and suitability. Smaller, faster models may suit classification, extraction, summarisation, routine transformation and high-volume processing; demanding tasks may need a more capable model.
Temperature	For APIs that support it, lower values generally produce more predictable output; higher values allow more variation. It is not a direct measure of truthfulness or accuracy.
Maximum completion tokens	A hard output limit, not a target length. Set it high enough for the expected response; split very long documents into sections.
Stop sequences	Terminate generation when specified text or tokens appear. Mainly useful in structured API workflows, generally unnecessary in ordinary chat.
Reasoning effort	Use defaults for routine work; increase effort for complex mathematics, constrained optimisation, difficult debugging, research synthesis or architecture decisions. Availability depends on model/API.

> Asking a model to “think harder” in a normal prompt does not itself change the underlying API configuration.

13. Golden Rules and Final Philosophy

25 golden rules of GPT-5.6 Luna prompting

1. Define the result before describing the process.

2. Make the objective specific and unambiguous.

3. Provide only relevant context.

4. Place important instructions before supporting material.

5. Use headings and delimiters for complex prompts.

6. Separate mandatory requirements from preferences.

7. Specify the desired output format.

8. Use measurable requirements instead of vague descriptions.

9. Start zero-shot; introduce examples only when necessary.

10. Decompose complex tasks when useful.

11. Avoid repeating instructions.

12. Do not prescribe unnecessary internal steps.

13. Define research sources and evidence requirements.

14. Distinguish verified facts, estimates and opinions.

15. State assumptions and uncertainty explicitly.

16. Request verification for important outputs.

17. Specify programming environments and restrictions.

18. Preserve existing functionality when modifying projects.

19. Require confirmation for consequential actions.

20. Define success criteria and stopping conditions.

21. Use follow-up prompts to refine results.

22. Keep prompts concise without sacrificing completeness.

23. Do not confuse confidence with verification.

24. Choose suitable model settings for demanding tasks.

25. Evaluate actual outputs and refine prompts based on observed results.

Final prompt engineering philosophy

The most effective way to prompt GPT-5.6 Luna is to communicate the desired outcome, supply the information that materially affects it, establish clear boundaries and define what success looks like.

> A weak prompt tells the model what topic to discuss. A good prompt tells it what task to accomplish. An excellent prompt explains the required outcome, why it matters, what information to use, what restrictions to follow, what evidence is necessary and how success will be evaluated.

Prompt engineering is a repeatable process: define objectives, communicate constraints, evaluate results and refine instructions. The goal is not to control every action or create increasingly complicated prompts; it is to provide enough information for a correct result while respecting requirements.

UNDERSTAND → DEFINE → CONTEXTUALISE → EXECUTE → VERIFY → REFINE.